

Pensamento Lógico Computacional com Python

Aula 03

Prof.ª Larissa

Sumário

- Estruturas de controle
 - while
 - for
- Estruturas de dados
 - Listas
 - for percorrendo listas

Estruturas de controle: repetição (while)

Há casos em que é preciso que um bloco de comandos seja executado várias vezes seguidas. Para isso, precisamos de uma estrutura de controle de repetição. O principal comando de repetição é o **while** (enquanto, em inglês).

A forma geral do while é

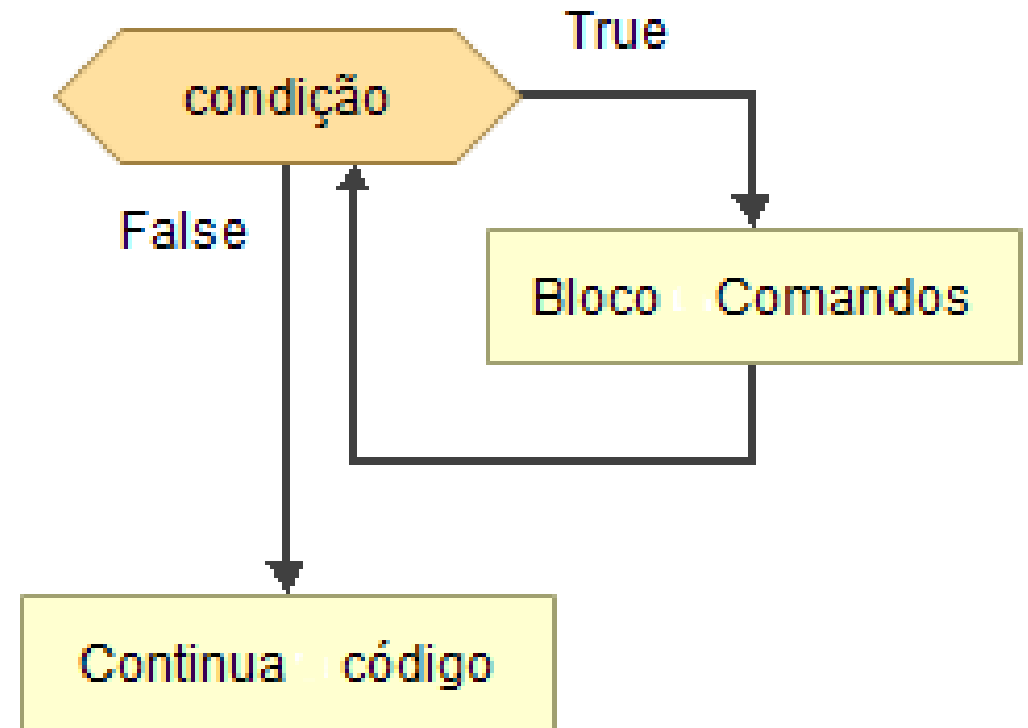
```
while condição:  
    bloco de comandos
```

Indentação
obrigatória!

O programa entende como

Falso: False, 0, *strings* ou estruturas vazias

Verdadeiro: True, valores $\neq 0$ e estr. ã vazias



Estruturas de controle: repetição (while)

Exemplo:

```
1. i = 5
2. while i>0:
3.     print(f"Contagem: {i}")
4.     i -= 1
5. print("Fim!")
```

```
Contagem: 5
Contagem: 4
Contagem: 3
Contagem: 2
Contagem: 1
Fim!
```

Execução	Condição	Saída print (3.)	Valor i (4.)
1	5 > 0 V	Contagem: 5	4
2	4 > 0 V	Contagem: 4	3
3	3 > 0 V	Contagem: 3	2
4	2 > 0 V	Contagem: 2	1
5	1 > 0 V	Contagem: 1	0
6	0 > 0 F		

Estruturas de controle: repetição (while)

Geralmente, o `while` é utilizado em casos em que **não se sabe** de antemão quantas vezes o bloco de comandos será executado, pois o `while` é bastante flexível para condições dinâmicas (que mudam a cada execução). Veja o exemplo.

```
1. resposta = "" #string vazia
2. while resposta != "sim" and resposta != "não":
3.     resposta = input("Digite 'sim' ou 'não': ").lower()
4. print("Resposta válida!")
```

No código acima, o laço de repetição será interrompido quando o usuário digitar uma das duas palavras: ou 'sim' ou 'não'. Em qualquer outra situação, o valor associado à variável `resposta` vai gerar condição verdadeira, e o laço se repete. Não sabemos quantas vezes o bloco de comandos será executado, já que isso depende de uma ação do usuário em tempo de execução.

Estruturas de controle: repetição (while com break)

Além da geração de condição falsa, outra forma de interromper a execução de um laço (*loop*) é executando o comando **break**. Uma vez executado, ele encerra o laço do qual faz parte, e faz com que o programa continue.

```
1.  numero_secreto = 7
2.
3.  while True:  #Loop infinito (condição sempre verdadeira)
4.      palpite = int(input("Adivinhe o número secreto (1-10): "))
5.
6.      if palpite == numero_secreto:
7.          print("Parabéns! Você acertou!")
8.          break  #Interrompe o loop
9.
10.     print("Errado! Tente novamente.")
11.
12. print("Fim do jogo!")
```

Estruturas de controle: repetição (for)

Nos casos em que sabemos de antemão quantas vezes precisamos executar o bloco de comandos, podemos utilizar o laço `for`. O `for` usa uma variável que percorre valores ou elementos, que chamaremos de `i`. É muito comum ele vir acompanhado da função `range()` ou percorrer estruturas, como *strings*.

A forma geral do `for` com `range()` é

```
for i in range(início, fim, passo):  
    bloco de comandos
```

A função `range()` estabelece o intervalo de valores assumidos por `i`, seguindo três critérios:

início (opcional): primeiro valor assumido por `i` (padrão = 0);

fim (obrigatório): quando `i` atingiria ou ultrapassaria esse valor, a execução do bloco para;

passo (opcional): estabelece uma mudança no valor de `i`. Por exemplo, passo 1 incrementa `i` em uma unidade, e passo -1 decrementa `i` em uma unidade (padrão = 1).

Estruturas de controle: repetição (for)

Exemplo:

```
1. for i in range(5, 0, -1):  
2.     print(f"Contagem: {i}")  
3. print("Fim!")
```

↓ ↓ ↓

```
Contagem: 5  
Contagem: 4  
Contagem: 3  
Contagem: 2  
Contagem: 1  
Fim!
```

Execução	Valor i	Saída print (2.)
1	5	Contagem: 5
2	4	Contagem: 4
3	3	Contagem: 3
4	2	Contagem: 2
5	1	Contagem: 1
6	0	

Estruturas de controle: repetição (for)

Podemos omitir argumentos na função `range()`. Nesse caso, o interpretador entende qual deles foi omitido pelo número de argumentos que recebe.

3 argumentos: **for** *i* **in** `range(início, fim, passo)`:
 bloco de comandos

2 argumentos: **for** *i* **in** `range(início, fim)`:
Assume `passo=1` bloco de comandos

1 argumento: **for** *i* **in** `range(fim)`:
Assume `passo=1` e bloco de comandos
 `início=0`

Estruturas de controle: repetição (for)

Exemplos:

```
1. #Gera números de 1 a 5 (o 6 não é incluído)
2. for numero in range(1, 6):
3.     print(f"Número: {numero}")
```

```
Número: 1
Número: 2
Número: 3
Número: 4
Número: 5
```

```
1. for i in range(3): #Repete 3 vezes (0, 1, 2)
2.     print(f"Repetição {i}")
```

```
Repetição 0
Repetição 1
Repetição 2
```

Estruturas de controle: repetição (for percorrendo *strings*)

Podemos utilizar o laço for para percorrer estruturas, como *strings*. Nesse caso, a variável assume cada um dos caracteres componentes. Observe os exemplos.

```
1. palavra = "Python"
2. for letra in palavra:
3.     print(letra)
```

P
y
t
h
o
n

```
1. palavra = "Abacaxi"
2. vogais = 0
3. for letra in palavra:
4.     #Verifica se a letra é uma vogal
5.     if letra in "aeiouAEIOU":
6.         vogais += 1
7. print(f"Total de vogais: {vogais}")
```

Total de vogais: 4

Estruturas de dados: Listas - introdução

Em muitas situações cotidianas, organizamos dados em uma lista: lista de compras, lista de disciplinas, lista de tarefas, e assim por diante. Em Python, podemos organizar elementos em uma estrutura de dados chamada **lista**. Uma lista é uma **coleção de objetos**, que podem ser de qualquer tipo: números, *strings*, ou até mesmo outras listas.

Sintaticamente, uma lista é representada como uma sequência de elementos, separados entre si por vírgula, representados dentro de colchetes. Por exemplo, podemos criar a lista *animais*, contendo três *strings*, com o comando:

```
>>> animais = ["peixe", "gato", "cão"]
```

Índice:	0	1	2
---------	---	---	---

Cada elemento da lista é representado por um índice, que começa em 0 e termina em $n-1$ (onde n é o número de elementos da lista).

Estruturas de dados: Listas - alguns operadores

Vamos testar alguns operadores e funções usados para manipular listas. A maior parte dos operadores de *strings* operam também sobre listas.

```
>>> animais = ["peixe", "gato", "cão"]
```

```
>>> animais
```

```
['peixe', 'gato', 'cão']
```

```
>>> animais[2]
```

```
'cão'
```

```
>>> animais + animais
```

```
['peixe', 'gato', 'cão', 'peixe', 'gato', 'cão']
```

```
>>> animais * 2
```

```
['peixe', 'gato', 'cão', 'peixe', 'gato', 'cão']
```

Estruturas de dados: Listas - alguns operadores

Continuando...

```
>>> 'peixe' in animais
```

```
True
```

```
>>> 'tartaruga' not in animais
```

```
True
```

```
>>> len(animais) #Número de elementos
```

```
3
```

```
>>> min(animais) #Menor valor em ordem numérica ou alfabética  
'cão'
```

```
>>> max(animais) #Maior valor em ordem numérica ou alfabética  
'peixe'
```

Estruturas de dados: Listas - alguns métodos

Os **métodos** de lista são funções chamadas pelo operador de ponto para operar sobre listas específicas. Veremos alguns, a seguir.

```
>>> numeros = [4, 5, 1, 3]
```

```
>>> numeros.append(1) #Adiciona o elemento 1 ao fim da lista
```

```
>>> numeros
```

```
[4, 5, 1, 3, 1]
```

```
>>> numeros.count(1) #Conta quantas vezes o elemento 1 apareceu
```

```
2
```

```
>>> numeros.remove(1) #Remove a 1ª ocorrência do elemento 1
```

```
>>> numeros
```

```
[4, 5, 3, 1]
```

```
>>> numeros.pop() #Remove o último elemento da lista e retorna
```

```
1
```

Estruturas de dados: Listas - alguns métodos

Continuando...

```
>>> numeros
```

```
[4, 5, 3]
```

```
>>> numeros.reverse() #Inverte a ordem dos elementos da lista
```

```
>>> numeros
```

```
[3, 5, 4]
```

```
>>> numeros.sort() #Organiza em ordem crescente ou alfabética
```

```
>>> numeros
```

```
[3, 4, 5]
```

```
>>> numeros.insert(1, 8) #Insere elemento 8 antes do índice 1 atual
```

```
>>> numeros
```

```
[3, 8, 4, 5]
```

Estruturas de dados: for percorrendo listas

Podemos utilizar o laço for para percorrer estruturas, como listas. Nesse caso, a variável assume cada um dos elementos componentes. Observe os exemplos.

```
1. lista_frutas = ["maçã", "uva", "morango", "romã"]
2. for fruta in lista_frutas:
3.     print(fruta.upper()) #Exibe c/ caixa alta
```

```
MAÇÃ
UVA
MORANGO
ROMÃ
```

```
1. numeros = [2, 3, 4, 5]
2. dobrados = [] #Cria lista vazia
3.
4. for num in numeros:
5.     dobrados.append(num*2)
6.
7. print(dobrados)
```

```
[4, 6, 8, 10]
```